COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	09/2/26	**DATE SUBMITTED:**	09/2/26
NAME:	Jiro Luis A. Katindig	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 6.1 **6.1 Searching Techniques**

1. PROCEDURE OUTPUT

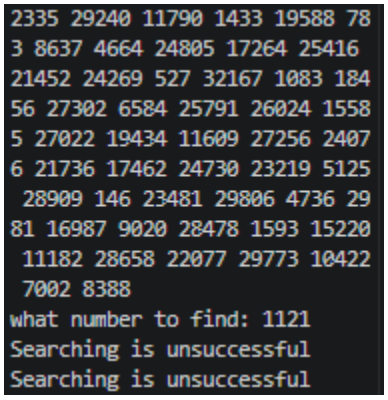
Screenshot	
Observations	The program generates a list of random numbers and asks the user to search for a value. When the user enters 1121, the program fails to find it in the list and outputs "Searching is unsuccessful". Because of redundant search loops in the code, the unsuccessful message ends up being printed twice.

Table 6-1. Data Generated and Observations.

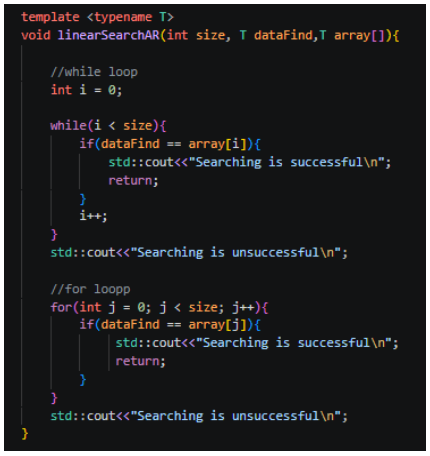
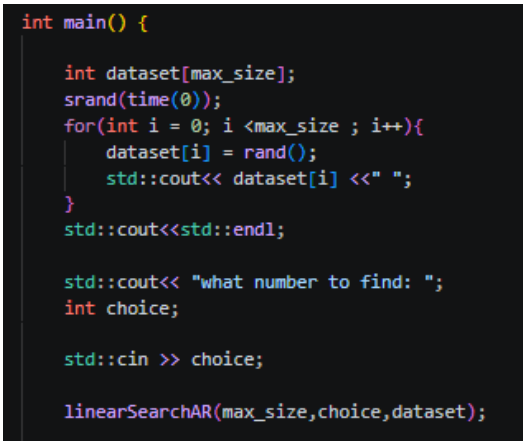
Code	 

Table 6-2. Code for the Searching Techniques

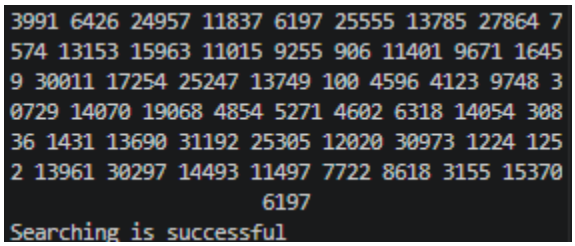
Output	
Observations	The code performs a linear search on a list of randomly generated numbers to check if the user's input exists. It successfully finds the target value 6197 within the dataset and displays "Searching is successful". However, the logic is redundant because the search function contains both a while loop and a for loop that perform the exact same check back-to-back, causing the program to scan the list twice if a number is missing.

Table 6-2a. Linear Search for Arrays

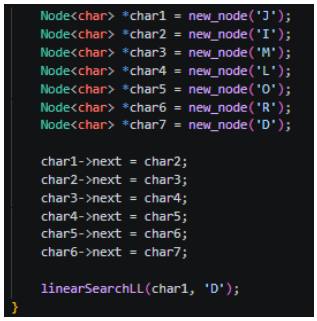
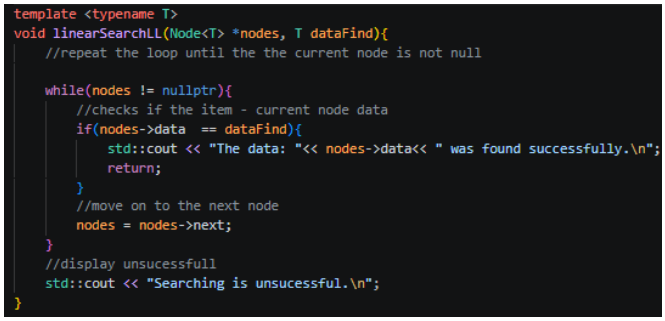
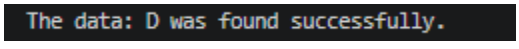
Code	 
Output	
Observations	The code performs a linear search on a single linked list to locate a specific character value. Seven individual character nodes storing the sequence 'J', 'I', 'M', 'L', 'O', 'R', and 'D' are manually instantiated and linked sequentially from char1 to char7. When the search function is called to find the value 'D', a traversal loop navigates through the linked list starting from the head node (char1). Once it reaches char7, the matching value is identified, printing "The data: D was found successfully." to the output and terminating the function. Overall, the program successfully traverses the list and finds the target value, though manually manually linking each node step-by-step is less scalable than dynamic list insertion.

Table 6-2b. Linear Search for Linked List

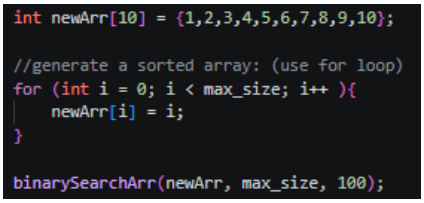
Code	 
Output	The data was not found
Observations	The code performs a binary search to check if the number 100 exists inside a sorted list of numbers. It works by repeatedly splitting the list in half to quickly pinpoint the target value. Since 100 is higher than any number in the generated list, the search narrows down to the end without finding a match, correctly displaying "The data was not found".

Table 6-3a. Binary Search for Arrays

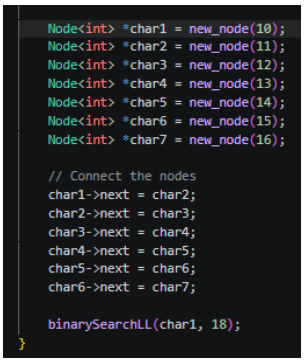
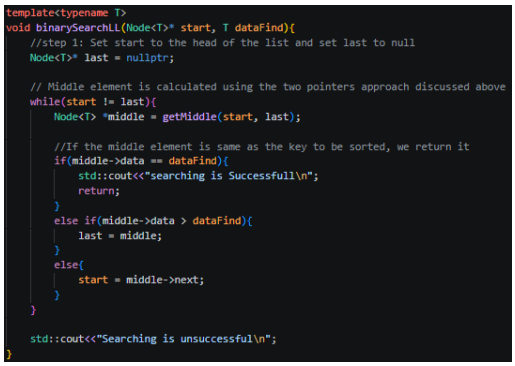
Code	 
Output	Searching is unsuccessful
Observations	The code performs a binary search on a connected chain of nodes storing numbers from 10 to 16 to find the target value 18. It repeatedly uses a helper function to locate the middle node in the current range and narrow down the search space. Because 18 is larger than any of the values in the linked chain, the search loop finishes without finding a match, correctly displaying "Searching is unsuccessful".

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code

```cpp
template <typename T>
int linearSearchARCount(T array[], int size, T dataFind, int &foundIndex){
    int comparisons = 0;
    for(int i = 0; i < size; i++){
        comparisons++;                      // one comparison happens here
        if(dataFind == array[i]){
            foundIndex = i;
            return comparisons;
        }
    }
    foundIndex = -1;
    return comparisons;
}

template <typename T>
int linearSearchLLCount(Node<T> *nodes, T dataFind, bool &found){
    int comparisons = 0;
    while(nodes != nullptr){
        comparisons++;                      // one comparison happens here
        if(nodes->data == dataFind){
            found = true;
            return comparisons;
        }
        nodes = nodes->next;
    }
    found = false;
    return comparisons;
}
```

```cpp
#include <iostream>
#include "searchingAlgo.h"

int main(){
    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;

    //  ARRAY APPROACH
    std::cout << "-- Array approach --\n";
    int foundIndex;
    int arrComparisons = linearSearchARCount(list, size, key, foundIndex);
    if(foundIndex != -1){
        std::cout << "Key " << key << " found at index " << foundIndex << "\n";
    } else {
        std::cout << "Key " << key << " not found\n";
    }
    std::cout << "Comparisons needed (array): " << arrComparisons << "\n\n";

    //  LINKED LIST APPROACH
    std::cout << "-- Linked list approach --\n";
    Node<int>* head = new_node(list[0]);
    Node<int>* current = head;
    for(int i = 1; i < size; i++){
        current->next = new_node(list[i]);
        current = current->next;
    }

    bool found;
    int llComparisons = linearSearchLLCount(head, key, found);
    std::cout << "Key " << key << (found ? " found in linked list\n" : " not found in linked list\n");
    std::cout << "Comparisons needed (linked list): " << llComparisons << "\n";

    return 0;
}
```

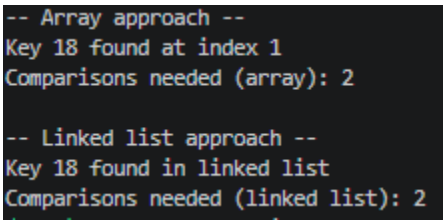
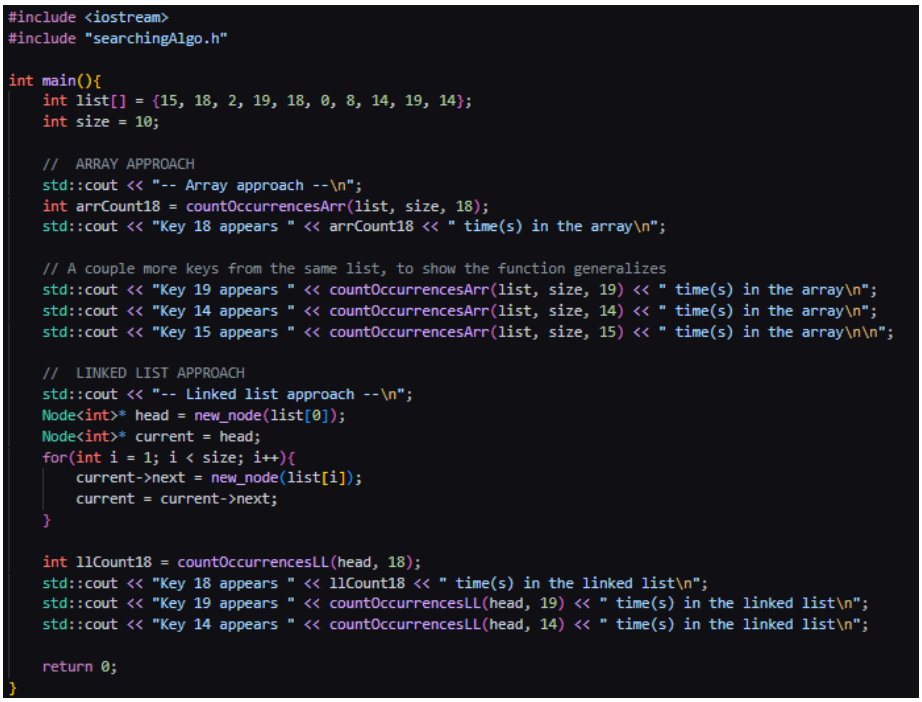
Output	
Observations	The code performs a linear search on both an array and a linked list to find the value 18 while tracking the total number of comparisons made. Since 18 is located at the second position in both data structures (index 1), both approaches successfully find the target value after making exactly 2 comparisons.

Table 1 Problem 1.

Code	

```cpp
#include <iostream>
#include "searchingAlgo.h"

int main(){
    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;

    //  ARRAY APPROACH
    std::cout << "-- Array approach --\n";
    int arrCount18 = countOccurrencesArr(list, size, 18);
    std::cout << "Key 18 appears " << arrCount18 << " time(s) in the array\n";

    // A couple more keys from the same list, to show the function generalizes
    std::cout << "Key 19 appears " << countOccurrencesArr(list, size, 19) << " time(s) in the array\n";
    std::cout << "Key 14 appears " << countOccurrencesArr(list, size, 14) << " time(s) in the array\n";
    std::cout << "Key 15 appears " << countOccurrencesArr(list, size, 15) << " time(s) in the array\n\n";

    //  LINKED LIST APPROACH
    std::cout << "-- Linked list approach --\n";
    Node<int>* head = new_node(list[0]);
    Node<int>* current = head;
    for(int i = 1; i < size; i++){
        current->next = new_node(list[i]);
        current = current->next;
    }

    int llCount18 = countOccurrencesLL(head, 18);
    std::cout << "Key 18 appears " << llCount18 << " time(s) in the linked list\n";
    std::cout << "Key 19 appears " << countOccurrencesLL(head, 19) << " time(s) in the linked list\n";
    std::cout << "Key 14 appears " << countOccurrencesLL(head, 14) << " time(s) in the linked list\n";

    return 0;
}
```

```cpp
template <typename T>
int countOccurrencesArr(T array[], int size, T dataFind){
    int occurrences = 0;
    for(int i = 0; i < size; i++){
        if(dataFind == array[i]){
            occurrences++;
        }
    }
    return occurrences;
}

template <typename T>
int countOccurrencesLL(Node<T> *nodes, T dataFind){
    int occurrences = 0;
    while(nodes != nullptr){
        if(nodes->data == dataFind){
            occurrences++;
        }
        nodes = nodes->next;
    }
    return occurrences;
}
```

Output

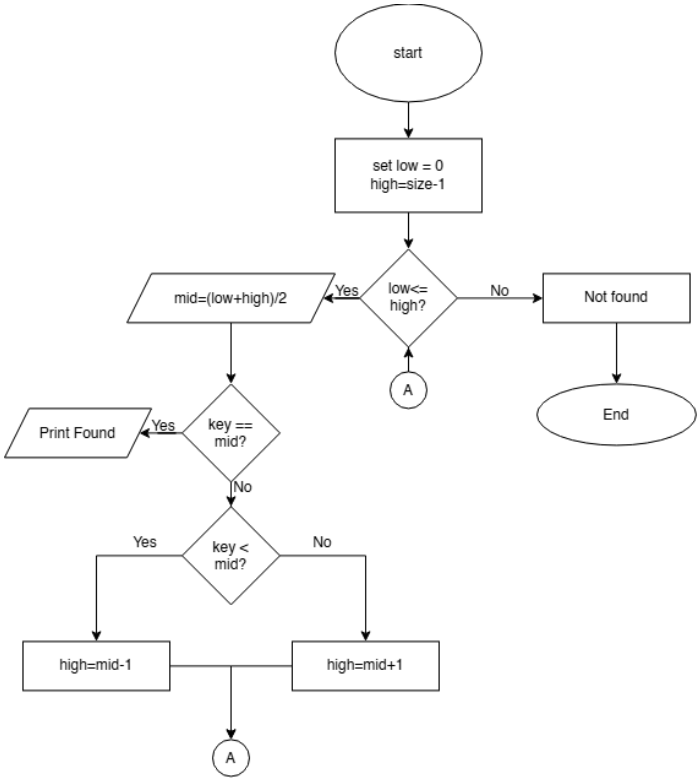
```
-- Array approach --
Key 18 appears 2 time(s) in the array
Key 19 appears 2 time(s) in the array
Key 14 appears 2 time(s) in the array
Key 15 appears 1 time(s) in the array

-- Linked list approach --
Key 18 appears 2 time(s) in the linked list
Key 19 appears 2 time(s) in the linked list
Key 14 appears 2 time(s) in the linked list
```

Observations

The code counts how many times specific numbers show up in both an array and a linked list by scanning through every element from start to finish. Both methods yield identical results, accurately reporting that keys 18, 19, and 14 appear 2 times each, while 15 appears 1 time, showing that both data structures process search occurrences identically.

Table 2 Problem 2.



Code

```
#include <iostream>
#include "searchingAlgo.h"

int main(){
    int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int key = 8;

    std::cout << "Finding key " << key << " \n\n";
    std::cout << "List (indices 0-9): 3 5 6 8 11 12 14 15 17 18\n\n";

    binarySearchArrVerbose(sortedList, size, key);

    return 0;
}
```

```cpp
template<typename T>
void binarySearchArrVerbose(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize - 1, middle;
    int iteration = 1;

    while(low <= high){
        middle = (int)((low + high) / 2);
        std::cout << "Iteration " << iteration << ": low=" << low
                  << " high=" << high << " middle=" << middle
                  << " -> data[middle]=" << data[middle] << "\n";

        if(dataFind == data[middle]){
            std::cout << "  Match! " << dataFind << " found at index "
                      << middle << "\n";
            std::cout << "The data was found\n";
            return;
        }
        else if(dataFind < data[middle]){
            std::cout << "  " << dataFind << " < " << data[middle]
                      << " -> search LEFT half\n";
            high = middle - 1;
        }
        else{
            std::cout << "  " << dataFind << " > " << data[middle]
                      << " -> search RIGHT half\n";
            low = middle + 1;
        }
        iteration++;
    }
    std::cout << "The data was not found\n";
}
```

Output

```
Finding key 8

List (indices 0-9): 3 5 6 8 11 12 14 15 17 18

Iteration 1: low=0 high=9 middle=4 -> data[middle]=11
  8 < 11 -> search LEFT half
Iteration 2: low=0 high=3 middle=1 -> data[middle]=5
  8 > 5 -> search RIGHT half
Iteration 3: low=2 high=3 middle=2 -> data[middle]=6
  8 > 6 -> search RIGHT half
Iteration 4: low=3 high=3 middle=3 -> data[middle]=8
  Match! 8 found at index 3
The data was found
```

Observations

The program uses a binary search algorithm to locate the value 8 inside a sorted list of ten numbers, tracing each step of the search process. By repeatedly checking the middle element and narrowing the range to either the left or right half, the code successfully finds 8 at index 3 on the fourth iteration and prints "The data was found". Additionally, a minor mistake in the flowchart sets high = mid + 1 instead of high = mid - 1 when searching the left half, though the program's code executes the logic correctly.

Table 3 Problem 3.

Code

```cpp
#include <iostream>
#include "searchingAlgo.h"

int main(){
    int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int size = 10;
    int key = 8;

    std::cout << "Finding key " << key << "\n\n";
    std::cout << "List (indices 0-9): 3 5 6 8 11 12 14 15 17 18\n\n";

    binarySearchArrRecursive(sortedList, 0, size - 1, key);

    return 0;
}
```

```cpp
template<typename T>
bool binarySearchArrRecursive(T data[], int low, int high, T dataFind, int iteration = 1){
    if(low > high){
        std::cout << "The data was not found\n";
        return false;
    }

    int middle = (int)((low + high) / 2);
    std::cout << "Iteration " << iteration << ": low=" << low
              << " high=" << high << " middle=" << middle
              << " -> data[middle]=" << data[middle] << "\n";

    if(dataFind == data[middle]){
        std::cout << "  Match! " << dataFind << " found at index "
                  << middle << "\n";
        std::cout << "The data was found\n";
        return true;
    }
    else if(dataFind < data[middle]){
        std::cout << "  " << dataFind << " < " << data[middle]
                  << " -> search LEFT half\n";
        return binarySearchArrRecursive(data, low, middle - 1, dataFind, iteration + 1);
    }
    else{
        std::cout << "  " << dataFind << " > " << data[middle]
                  << " -> search RIGHT half\n";
        return binarySearchArrRecursive(data, middle + 1, high, dataFind, iteration + 1);
    }
}
```

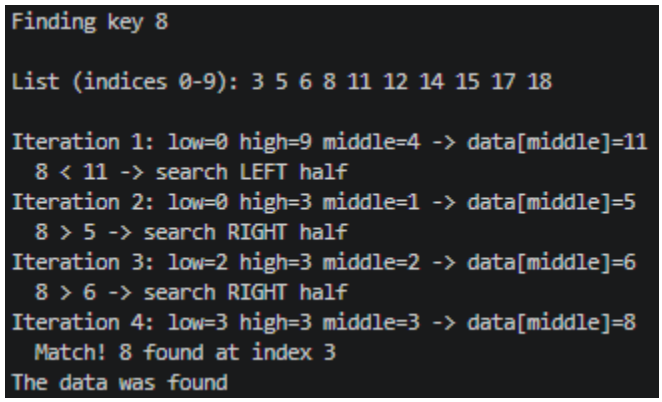
Output	
Observations	The program uses a recursive binary search function to locate the target value 8 inside a sorted list of ten numbers. Instead of using a standard loop, the function repeatedly calls itself with updated boundary parameters (low or high) to narrow the search range in half during each step. The output traces the recursion through four iterations until it matches 8 at index 3, successfully printing "The data was found".

Table 4 Problem 4.

CONCLUSION

Overall, this activity was easily one of the most exhausting ones I've done so far, but it really showed how linear search and binary search handle things differently when you're working with arrays versus linked lists.

Tracing all those loops, pointers, and recursive calls step-by-step was honestly a total headache, especially when trying to spot minor code bugs like double printing or flowchart typos. Linear search is simple enough since it works on literally anything whether it's sorted or not, But scanning through items one by one getspretty tedious and inefficient real quick, especially when you're counting every single comparison manually.

On the flip side, binary search is way faster since it keeps cutting the search space in half, but setting it up is super picky. Your data *has* to be sorted beforehad, and using it on linked lists is a whole extra level of annoying because you can't just jump to an index, you have to mess around with slow and fast pointers just to find the middle node.

At the end of the day, as painful as it was to write out all the observations and trace every single iteration, it made the trade-offs pretty clear. linear search is fine for quick, messy, or unsorted lists, but if you want actual speed on bigger data, binary search is worth the extra setup hassle.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.